International Journal of Information Security (2024) 23:3609–3623
https://doi.org/10.1007/s10207-024-00898-w

REGULAR CONTRIBUTION

Secure multi-party computation with legally-enforceable fairness

Takeshi Nakai[1] · **Kazumasa Shinagawa**[2,3]

Published online: 29 August 2024
© The Author(s) 2024

Abstract
Fairness is a security notion of secure computation and cannot always be achieved if an adversary corrupts a majority of parties in standard settings. Lindell (CT-RSA 2008) showed that imposing a monetary penalty on an adversary can circumvent the impossibility. He formalized such a security notion as "legally enforceable fairness" for the *two-party* setting based on the ideal trusted bank functionality and showed a protocol achieving the requirements. Based on the same framework, we introduce secure *multi-party* computation with legally enforceable fairness that is applicable for an arbitrary number of parties. Further, we propose two protocols that realize our introduced functionality. The first one achieves $O(n)$ rounds and $O(n\alpha)$ fees, where n is the number of parties, and α is a parameter for the penalty amount. The fee refers to the balance amount in the bank required at the beginning of the protocol, which evaluates the difficulty of participating in the protocol in a financial sense. The second one achieves $O(1)$ rounds and $O(n^2\alpha)$ fees.

1 Introduction

1.1 Backgrounds

Secure computation is a cryptographic protocol to enable distrustful parties to compute a function on their private inputs jointly [14, 30]. Fairness is a security notion of secure computation, which requires that at the end of a protocol, either all parties learn the output or none of them learn it. Fairness implies that no adversary can abort the protocol without telling the output to honest parties. Unfortunately, for many functions, fairness is impossible to achieve in the plain model without a majority of honest parties [2, 10, 16].

There are works to circumvent the impossibility result. One of the works is the gradual release approach [6, 15]. In this approach, parties gradually disclose the output with multiple rounds instead of revealing it at once. It achieves fairness substantially since there is little difference in knowledge of the output value with an honest party, even if an adversary aborts. However, this solution has the drawback of requiring many rounds. Another approach is the optimistic model [3, 24] that uses a Trusted Third Party (TTP). Although the TTP does not appear in the protocol if all parties behave honestly, it works to restore fairness when an adversary violates fairness. This solution is efficient; however, it has the drawback of relying on the honesty of the third party.

Lindell [23] introduced a new approach to achieving fairness, which is a variant of the optimistic model. The new paradigm called *legally enforceable fairness*, guarantees that an adversary who violates fairness is imposed a monetary penalty, and an honest party who does not learn the output receives monetary compensation. If the penalty amount is determined appropriately, we can achieve fairness with this approach since adversaries refrain from aborting to avoid losing money and follow procedures. Lindell formalized secure *two-party* computation with legally enforceable fairness based on a trusted bank, which corresponds to TTP in the optimistic model, and showed a two-party protocol for any functionality. The bank manages all parties' accounts and can update their balances. Further, parties can request that the bank updates their balances by submitting an electronic cheque.

1.2 Related works

Bentov and Kumaresan [7] introduced a functionality that achieves fairness with monetary penalties applicable to an arbitrary number of parties. However, their work uses Bitcoin [28] instead of the trusted bank. Blockchain-based cryptocur-

✉ Takeshi Nakai
nakai@cs.tut.ac.jp

1 Toyohashi University of Technology, Aichi, Japan

2 Ibaraki University, Ibaraki, Japan

3 National Institute of Advanced Industrial Science and Technology, Tokyo, Japan

rencies, such as Bitcoin, have the advantage of not relying upon TTP. Thus, the cryptocurrency-based solution can avoid the TTP-dependent problem of the bank-based one. Against such a background, cryptocurrency-based solutions dominate in works to achieve fairness with monetary penalties, e.g., [1, 4, 5, 7, 8, 11, 13, 18, 20–22, 25, 26, 29].

However, cryptocurrency-based protocols also have some disadvantages. For cryptocurrencies that do not guarantee the anonymity, parties are forced to publicly disclose a part of the protocol since the blockchain is a public data structure. This can be a drawback for a party who does not want to disclose his/her participation in the protocol. Another issue is due to the double spending attack on cryptocurrencies. To counter the attack, parties must wait in a step with a certain time for mining process. For cryptocurrencies where the mining process takes a large amount of time, this can be a drawback in terms of protocol efficiency. Indeed, Bentov and Kumaresan [7] claim that parties should use about an hour per round on their protocol based on Bitcoin.

1.3 Our contribution

We introduce legally enforceable fairness applicable to an arbitrary number of parties. It guarantees that every honest party can receive monetary compensation if the protocol terminates when only an adversary learns the output. We note that guaranteeing that all honest parties receive compensation implicitly requires the adversary to lose money. It is a natural generalization of Lindell's formalization for the two-party setting.

We show secure multi-party computation protocols with legally enforceable fairness. We evaluate the efficiency of a protocol by round complexity and *the amount of fee to participate in the protocol*. More specifically, the fee refers to the initial amount for which any (honest party's) account balance does not become negative, i.e., less than zero, in any round. (Note that although the legally enforceable fairness assures every honest party's account balance will not turn into a loss at the end of the protocol, it could be lost temporarily during the protocol.) The fee amount measures the wealth difficulty of participating in the protocol, which is introduced in the work of the Bitcoin-based protocol [26].

We propose two protocols: The first one achieves $O(n)$ rounds and $O(n\alpha)$ fees, where n is the number of parties, and α is a parameter for the penalty amount. The second one achieves $O(1)$ rounds and $O(n^2\alpha)$ fees.

The electronic cheque formalized by Lindell [23] has no field to specify an expiration time. Because of this, parties must hold their cheques *indefinitely* to reap legally enforceable fairness. This paper further tackles this issue. We introduce a new trusted bank that can handle electronic cheques with expiration times and show how to apply the bank to our protocols.

Remark 1 We formalize fairness with monetary penalties for the multi-party setting, as in Bentov and Kumaresan's work [7]. However, since their model differs from ours, their formalization for the multi-party setting also differs. For instance, cryptocurrency-based protocols require parties to explicitly input coins (money) into the protocols as deposits. It is because parties need to create transactions on the public network specifying the coins they use. On the other hand, our formalization does not require such inputs, as well as [23], since the bank handles all monetary operations implicitly.

1.4 Differences from the conference version

We summarize differences from the earlier version of this paper [27] below:

(1) *Proposal of new trusted bank functionality*: We introduce a new trusted bank functionality that handles electronic cheques with expiration time. Furthermore, we present multi-party protocols with legally enforceable fairness based on the new bank functionality. (It corresponds to Sect. 5.)
(2) *Security proof for our second protocol*: We present a security proof for our second protocol. (It corresponds to Appendix B.)

Besides, we improve some descriptions of the proposed protocols and the security proof for our first protocol.

1.5 Organization

The remaining part of this paper is organized as follows: In Sect. 2, we introduce basic notations, a security model, and ideal functionalities. Section 3 describes Lindell's two-party protocol with legally enforceable fairness [23]. Section 4 introduces secure multi-party computation with legally enforceable fairness and proposes two protocols as described above. In Sect. 5, we newly introduce a cheque with an expiration time and modified bank functionality according to it. Also, this section shows how to apply the modified cheque and bank to our proposed protocols. We conclude this paper in Sect. 6. (Sect. 5 and Appendix B are newly added from the earlier version [27].)

2 Preliminaries

2.1 Basic notations

For any positive integer i, we define $[i] := \{1, \ldots, i\}$. For a finite set X, $x \in_R X$ means the process of choosing an element $x \in X$ uniformly at random.

We denote by n and λ the number of parties and the security parameter, respectively. Let $H \subseteq [n]$ be the set of honest parties and let $C \subsetneq [n]$ be the set of corrupted parties controlled by an adversary. The sets satisfy $h + c = n$, where $h := |H|$ and $c := |C|$ since each party is either honest or corrupted. We consider settings where $c < n$. We assume that all parties are non-uniform Probabilistic Polynomial-Time (PPT) algorithms in λ.

2.2 Public-key infrastructure

Our protocol assumes the existing of a public-key infrastructure, as well as [23]. We define the infrastructure as in Functionality 1 that holds the basic abilities of key registrations and retrievals. This definition is according to the formalization of certificate authority of [9].

2.3 Trusted bank functionality

Assume that a trusted bank manages all parties' accounts, and it has the authority to update their balances. A party can request the bank to update balances by submitting a digital cheque. We define the cheque in the following notation.

Cheque. A cheque requesting payment of $\$q$ from P_i to P_j is a signed message of the form $\mathsf{chq}(cid, i \to j, q, z)$, where cid is a unique identifier and z is an auxiliary information field. We say a cheque is valid when the cheque consists of the elements $cid, i \to j, q$, and z and all of them are signed with P_i's signing key.

Based on the cheque, we define a functionality that represents a trusted bank as in Functionality 2. Let $\mathsf{bal}[i]$ be a variable of the current balance of P_i for $i \in [n]$. We allow an account balance to be negative for the sake of simplicity. (This assumption implies that, even if there is not a sufficient balance in a corrupted party's account, honest parties can still take compensation from it.)

In the execution phase, $\mathcal{F}_{\text{bank}}$ updates the balances according to a cheque sent from a party. Upon receiving a cheque $\mathsf{chq}(cid, i \to j, q, z)$, the functionality confirms the validity and sets balances as $\mathsf{bal}[i] = \mathsf{bal}[i] - q$ and $\mathsf{bal}[j] = \mathsf{bal}[j] + q$. After that, it sends P_i a copy of the cheque. We use a set used to prevent duplicate usages of a cheque.

2.4 Secure computation with Abort

Let $\mathcal{F}$ be a PPT n-party ideal functionality and let π be a PPT protocol for computing $\mathcal{F}$. We follow the real/ideal paradigm as a security notion. Informally, in the ideal world, parties send their inputs to $\mathcal{F}$ that first replies to the output to a simulator. The simulator can choose whether or not to abort the protocol by replying fair or unfair to $\mathcal{F}$. If it replies fair, all parties learn the output, and the functionality terminates. Otherwise, the functionality terminates without sending the output to honest parties. Namely, this ideal world allows the simulator to violate fairness.

Let $\text{IDEAL}_{\mathcal{F},\mathcal{S}}(\lambda, z)$ denote the output vector of honest parties and a simulator $\mathcal{S}$ (with an auxiliary input z) in the ideal world for realizing $\mathcal{F}$. Let $\text{HYBRID}^{\mathcal{G}}_{\pi,\mathcal{A}}(\lambda, z)$ denote the output vector of honest parties and an adversary $\mathcal{A}$ (with an auxiliary input z) in the real (hybrid) world for executing a hybrid protocol π with an ideal functionality $\mathcal{G}$.

Definition 1 We say that a protocol π securely computes $\mathcal{F}$ with abort in the $\mathcal{G}$ hybrid model if for every non-uniform PPT adversary $\mathcal{A}$, there exists a non-uniform PPT simulator $\mathcal{S}$ such that two families of probability distributions $\{\text{IDEAL}_{\mathcal{F},\mathcal{S}}(\lambda, z)\}_{\lambda\in\mathbb{N},z\in\{0,1\}^*}$ and $\{\text{HYBRID}^{\mathcal{G}}_{\pi,\mathcal{A}}(\lambda, z)\}_{\lambda\in\mathbb{N},z\in\{0,1\}^*}$ are computationally indistinguishable.

3 Existing protocol for two-party setting

In this section, we introduce Lindell's protocol [23] for secure two-party computation with legally-enforceable fairness, on which our protocols are based.

Functionality 1 The certificate authority functionality $\mathcal{F}_{\text{CA}}$

1. Upon receiving a message $(\mathsf{Register}, sid, v)$ from a party P, send $(\mathsf{Register}, sid, v)$ to a simulator. Upon receiving back ok from the simulator, then check if $sid = P$ and if this registration is the first request from P. If both checks are passed, then record the pair (P, v); otherwise, ignore the message.
2. Upon receiving a message $(\mathsf{Retrieve}, sid)$ from P, send $(\mathsf{Retrieve}, sid)$ to a simulator. Upon receiving back ok from the simulator, if there is a record (sid, v), send $(\mathsf{Retrieve}, sid, v)$ to P. Otherwise, send $(\mathsf{Retrieve}, sid, \perp)$ to P.

Functionality 2 The trusted bank functionality $\mathcal{F}_{\text{bank}}$

Running with $\mathcal{F}_{\text{CA}}$ and parties $P_1, \ldots, P_n$. Let $\mathsf{bal}[u]$ $(u \in [n])$ be a variable denoting the current balance of P_u.

Setup phase: Initialize used as an empty set. Send $(\mathsf{Retrieve}, u)$ to $\mathcal{F}_{\text{CA}}$ for every $u \in [n]$ and wait to the response. If any response has $\perp$, then terminates the functionality. Stores the keys $\{vk_u\}_{u\in[n]}$, where vk_u is the verification key retrieved for P_u.

Execution phase: Upon receiving a cheque $\mathsf{chq}(cid, i \to j, q, z)$, perform the following process.

1. Check if the cheque is valid and if $(cid, i, j) \notin \mathsf{used}$.
2. If both checks are passed, set $\mathsf{bal}[i] = \mathsf{bal}[i] - q$ and $\mathsf{bal}[j] = \mathsf{bal}[j] + q$. Otherwise, ignore the cheque.
3. Set $\mathsf{used} \leftarrow \mathsf{used} \cup \{(cid, i, j)\}$.
4. Send the cheque to P_i.

Functionality 3 Secure two-party computation with legally enforceable fairness $\mathcal{F}^{\alpha}_{2,f}$

Running with parties P_1 and P_2, and a simulator $\mathcal{S}$ that corrupts P_j, where $j \in \{1,2\}$. Let P_i denote the honest party ($i \in \{1,2\}, i \neq j$). $\mathsf{bal}[i]$ and $\mathsf{bal}[j]$ denote the current balances of P_i and P_j, respectively.

Input phase: Wait to receive inputs x_i and x_j from P_i and $\mathcal{S}$, respectively. If either of the inputs is invalid, then send $\perp$ to both parties and terminate the functionality.

Output phase: Compute $y \leftarrow f(x_1, x_2)$, and send y to $\mathcal{S}$. Wait for a response from $\mathcal{S}$.

- If $\mathcal{S}$ replies fair, then send y to P_i.
- If $\mathcal{S}$ replies unfair, then set $\mathsf{bal}[j] = \mathsf{bal}[j] - \alpha$ and $\mathsf{bal}[i] = \mathsf{bal}[i] + \alpha$.

3.1 Ideal functionality for secure two-party computation

Before describing the protocol, we introduce the ideal functionality of secure two-party computation with legally-enforceable fairness [23]. Let α be a parameter for the amount of penalty and compensation. In principle, the functionality guarantees the following properties.

- No honest party loses money.
- If a corrupted party P_j aborts after learning the output and does not tell the value to the other party P_i, then P_j loses $\$\alpha$ and P_i obtains $\$\alpha$.

See Functionality 3 for a formal description of $\mathcal{F}^{\alpha}_{2,f}$ computing a function f. A simulator $\mathcal{S}$ corrupting P_j can obtain the output before an honest party P_i, and can choose whether or not to tell the value to P_i. In the case of telling the value to P_i (corresponding to the case where $\mathcal{S}$ responds fair), the functionality sends the output to P_i. In the other case (corresponding to the case where $\mathcal{S}$ responds unfair), the functionality imposes a financial penalty to P_j and compensates P_i instead of not telling the output to P_i.

3.2 Two-party protocol with legally enforceable fairness

Suppose parties P_1 and P_2 have inputs x_1 and x_2, respectively. Lindell's protocol consists of the main computation and output exchange phases. We describe an overview of the protocol below. (Suppose that both parties have registered their key pairs to a certificate authority before beginning of the main computation phase.)

Main computation phase: P_1 and P_2 run secure two-party computation with inputs x_1 and (x_2, r), respectively, where r is a random string to mask a cheque. As a result, P_1 receives a cheque $\mathsf{chq}_1 = \mathsf{chq}(cid, 2 \to 1, \alpha, r \oplus \mathsf{chq}_2)$, where $\mathsf{chq}_2 = \mathsf{chq}(cid, 1 \to 2, \alpha, y)$ and $y = f(x_1, x_2)$, and P_2 receives nothing. Both chq_1 and chq_2 are valid, i.e., both cheques are signed with signing keys of P_2 and P_1, respectively. Note that, although chq_1 includes chq_2 holding the output y, P_1 cannot learn the value since chq_2 is masked with r.

Output exchange phase: At the beginning of this phase, P_1 has chq_1 and P_2 has nothing. P_1 sends chq_1 to P_2. (P_2 can verify the validity of the received cheque using P_1's verification key.) P_2 unmasks the auxiliary information field and obtains chq_2. Since chq_2 has the output y in the auxiliary information field, P_2 learns the value first. Afterward, P_2 tells y to P_1 by revealing chq_2, and the protocol is finished. (P_1 can verify the output validity by verifying the validity of chq_2.)

As seen in the overview, $\mathcal{F}_{\mathrm{bank}}$ does not appear when both parties behave honestly. In the following, we explain the role of the bank in two cases where P_1 is corrupted or P_2 is corrupted.

The case where P_1 is corrupted: Let us consider the case where corrupted P_1 sends chq_1 to $\mathcal{F}_{\mathrm{bank}}$ without sending it to P_2. Since the cheque is valid, the bank sets $\mathsf{bal}[2] = \mathsf{bal}[2] - \alpha$ and $\mathsf{bal}[1] = \mathsf{bal}[1] + \alpha$, i.e., honest P_2 loses $\$\alpha$. However, since P_2 receives a copy of chq_1 from the bank, he/she can learn the output y and get back $\$\alpha$ using chq_2. Further, since P_1 receives a copy of chq_2, he/she also learns the output. This case satisfies fairness since both parties learn the output, and both balances are unchanged from the initial state.

The case where P_2 is corrupted: Let us consider the case where corrupted P_2 aborts the protocol without revealing y to P_1. Then, honest P_1 submits chq_1 to $\mathcal{F}_{\mathrm{bank}}$ and obtains $\$\alpha$. To get back the money, P_2 must send chq_2 to $\mathcal{F}_{\mathrm{bank}}$. If P_2 sends the cheque to the bank, P_1 learns the output from the copy, and this case satisfies fairness. Otherwise, P_1 obtains $\$\alpha$ as compensation instead not of learning y. This case also satisfies fairness.

4 Secure multi-party computation with legally enforceable fairness

This section shows our secure multi-party protocols with legally enforceable fairness. We propose two protocols: The first one achieves $O(n)$ rounds and $O(n\alpha)$ fees. The second one achieves $O(1)$ rounds and $O(n^2\alpha)$ fees. The two protocols are inspired by the Bitcoin-based protocols [7] and [25], respectively.

Overview of our protocols: We construct n-party protocols following Lindell's construction for two-party protocol [23], that is, our protocols also consist of the main computation and the output exchange phases. It is well known that the ideal oblivious transfer $\mathcal{F}_{\mathrm{OT}}$ is sufficient to achieve secure computation for arbitrary functionality according to Definition 1 [17, 19]. Moreover, this can be performed in constant

rounds [17]. The main computation phases in both protocols are performed in constant rounds in the $\mathcal{F}_{\mathrm{OT}}$-hybrid model. Afterward, parties run an output exchange protocol based on the ideal bank functionality. Hence, we design protocols in the $(\mathcal{F}_{\mathrm{OT}}, \mathcal{F}_{\mathrm{CA}}, \mathcal{F}_{\mathrm{bank}})$-hybrid model. We note that there is almost no difference in efficiency of the main computation phases between the two protocols. The differences occur within the output exchange phase.

Main difference from the Bitcoin-based protocols: The main difference between Bitcoin-based protocols [7, 26] and our protocols is that the former uses *claim-or-refund* functionality, while the latter uses the trusted bank functionality. The claim-or-refund functionality allows a sender to transfer coins to a receiver if the receiver publishes a witness that satisfies a condition. The condition can be any predicate specified by the sender. On the other hand, the trusted bank functionality transfers money to the sender's account if the receiver sends a cheque with a valid signature of the sender to the bank. Note that only signature verification can be set as a condition for money transfer. Because of this difference, we cannot use the same strategy as Bitcoin-based protocols, and so we construct our protocols by extending Lindell's method to include another cheque in the auxiliary information field. This is the main difference from Bitcoin-based protocols.

Functionality 4 Secure multi-party computation with legally enforceable fairness $\mathcal{F}_{n,f}^{\geq\alpha}$

Running with parties $P_1, \ldots, P_n$ and a simulator $\mathcal{S}$ that corrupts parties $\{P_j\}_{j\in C}$. $\mathsf{bal}[i]$ denotes the current balance of P_i for $i \in [n]$.

Input phase: Wait to receive the following messages.

- (input, x_i) from P_i for $i \in H$
- $(\mathsf{input}, \{x_j\}_{j\in C}, H', \{\alpha_i\}_{i\in H'}, \{\beta_j\}_{j\in C})$ from $\mathcal{S}$, where $H' \subseteq H$, β_j is a non-negative integer, and $\sum_{i\in H'} \alpha_i = \sum_{j\in C} \beta_j$.

If some input is invalid, or some α_i is less than α, then send $\perp$ to all parties and terminate the functionality.

Output phase: Compute $y \leftarrow f(x_1, \ldots, x_n)$.

1. Perform the following process depending on h', where $h' := |H'|$.
   - If $h' = 0$, then send y to P_i for $i \in [n]$, and terminate.
   - If $0 < h' < h$, then set $\mathsf{bal}[i] = \mathsf{bal}[i] + \alpha_i$ for all $i \in H'$ and $\mathsf{bal}[j] = \mathsf{bal}[j] - \beta_j$ for $j \in C$, and terminate.
   - If $h' = h$, then send message y to $\mathcal{S}$.
2. In the case of $h' = h$, wait to $\mathcal{S}$'s response.
   - If $\mathcal{S}$ replies fair, send y to P_i for $i \in H$.
   - If $\mathcal{S}$ replies $(\mathsf{unfair}, H'', \{\alpha'_i\}_{i\in H}, \{\beta'_j\}_{j\in C})$ from $\mathcal{S}$, perform the following process, where $H'' \subsetneq H$, β'_j is a non-negative integer, $\alpha'_i \geq \alpha$ for $i \in H \setminus H''$, and $\sum_{i\in H} \alpha'_i = \sum_{j\in C} \beta'_j$. When $\mathcal{S}$ submits an invalid value, continue with $\alpha'_i = \alpha_i$ for $i \in H$ and $\beta'_j = \beta_j$ for $j \in C$.
     - Send y to P_i for $i \in H''$.
     - Set $\mathsf{bal}[i] = \mathsf{bal}[i] + \alpha'_i$ for $i \in H$ and $\mathsf{bal}[j] = \mathsf{bal}[j] - \beta'_j$ for $j \in C$.

Protocol 5 Multi-party protocol with legally enforceable fairness: $O(n)$ rounds and $O(n\alpha)$ fees

Parties $P_1, \ldots, P_n$ have $x_1, \ldots, x_n$ as inputs, respectively.

Registration phase: Every party generates a fresh key pair and registers the verification key to $\mathcal{F}_{\mathrm{CA}}$.

Main computation phase: Parties run a secure multi-party computation protocol as follows:

Input: P_1 inputs (cid_1, x_1). For $i \in \{2, \ldots, n-1\}$, P_i inputs (cid_i, x_i, r_i). P_n inputs $(cid_n, x_n, \{r_{n,j}\}_{j\in[n-1]})$. For each $i \in [n]$, $cid_i \in_{\mathrm{R}} \{0,1\}^\lambda$. Each of $r_2, \ldots, r_{n-1}, r_{n,1}, \ldots, r_{n,n-1}$ is a uniformly random string of appropriate length to achieve the property of the following output.

Output: P_1 receives a cheque chq_2^1 defined as follows:

- $y := f(x_1, \ldots, x_n)$;
- $cid := cid_1 \parallel \cdots \parallel cid_n$;
- $\mathsf{chq}_j^n := \mathsf{chq}(cid, j \to n, \alpha, y)$ for $j \in [n-1]$;
- $\mathsf{chq}_n^{n-1} := \mathsf{chq}(cid, n \to n-1, (n-1)\alpha, \{r_{n,j} \oplus \mathsf{chq}_j^n\}_{j\in[n-1]})$;
- $\mathsf{chq}_{i+1}^i := \mathsf{chq}(cid, (i+1) \to i, i\alpha, r_{i+1} \oplus \mathsf{chq}_{i+2}^{i+1})$ for $i \in [n-2]$;

That is, $\mathsf{chq}_2^1 = \mathsf{chq}(cid, 2 \to 1, \alpha, r_2 \oplus \mathsf{chq}_3^2)$, $\mathsf{chq}_3^2 = \mathsf{chq}(cid, 3 \to 2, 2\alpha, r_3 \oplus \mathsf{chq}_4^3)$, and so on. The other parties receive nothing.

Output exchange phase (all parties behave honestly):

1. For $i = 1$ to $n-1$, P_i sends chq_{i+1}^i to P_{i+1}, and P_{i+1} unmasks the string written in the auxiliary information field of the received cheque.
2. P_n tells y to P_i for all $i \in [n-1]$ by sending chq_i^n to P_i.

Output exchange phase (some parties behave maliciously):

- If a corrupted P_j $(2 \leq j \leq n)$ aborts the protocol. (It includes the case where an adversary tells y to only some of honest parties.)
  - For all $i \in [j-1]$, P_i submits chq_{i+1}^i to $\mathcal{F}_{\mathrm{bank}}$.
- If a corrupted P_j submits the cheque(s) to $\mathcal{F}_{\mathrm{bank}}$.
  - For $i \in [j-1]$, P_i submits chq_{i+1}^i to $\mathcal{F}_{\mathrm{bank}}$.
  - For $i = j+1$ to n, P_i receives a copy of chq_i^{i-1} from $\mathcal{F}_{\mathrm{bank}}$ and submits the cheque(s) obtained from the auxiliary information field.

4.1 Ideal functionality for secure multi-party computation

Before presenting our protocols, we introduce an ideal functionality for the multi-party setting. Our formalization is inspired by [7, 25]. In terms of generalizing Functionality 3, the functionality for multi-party setting should guarantee the following properties.

- No honest party loses money.
- If an adversary aborts after learning the output without telling the value to honest parties, then every honest party receives $\$\alpha$ *or more* as compensation.

Note that the second item does not guarantee that honest parties get compensation if an adversary aborts without

Protocol 6 Multi-party protocol with legally enforceable fairness: $O(1)$ rounds and $O(n^2\alpha)$ fees

Parties $P_1, \ldots, P_n$ have $x_1, \ldots, x_n$ as inputs, respectively. Below, let $M := \{2, \ldots, n-1\}$.

Registration phase: Every party generates a fresh key pair and registers the verification key to $\mathcal{F}_{\mathrm{CA}}$.

Main computation phase: Parties run a secure multi-party computation protocol as follows.

Input: P_1 inputs (cid_1, x_1). For $i \in M$, P_i inputs (cid_i, x_i, r_i, r'_i). P_n inputs $(cid_n, x_n, \{r_{n,j}\}_{j\in[n-1]})$. For each $i \in [n]$, $cid_i \in_{\mathrm{R}} \{0,1\}^\lambda$. Each of $r_2, \ldots, r_{n-1}, r'_2, \ldots, r'_{n-1}, r_{n,1}, \ldots, r_{n,n-1}$ is a uniformly random string of appropriate length to achieve the property of the following output.

Output: P_1 receives cheques $\{\mathsf{chq}_i^1\}_{i\in M}$ and $r'_2 \oplus \cdots \oplus r'_{n-1} \oplus \mathsf{chq}_n^1$ defined as follows:

- $y := f(x_1, \ldots, x_n)$;
- $cid := cid_1 \parallel \cdots \parallel cid_n$;
- $\mathsf{chq}_i^1 := \mathsf{chq}(cid, i \to 1, (n-2)\alpha, r_i \oplus \mathsf{chq}_1^i)$ for $i \in M$;
- $\mathsf{chq}_1^i := \mathsf{chq}(cid, 1 \to i, (n-1)\alpha, r'_i)$ for $i \in M$;
- $\mathsf{chq}_n^1 := \mathsf{chq}(cid, n \to 1, (n-1)\alpha, \{r_{n,i} \oplus \mathsf{chq}_i^n\}_{i\in[n-1]})$;
- $\mathsf{chq}_i^n := \mathsf{chq}(cid, i \to n, \alpha, y)$ for $i \in [n-1]$

The other parties receive nothing.

Output exchange phase (all parties behave honestly):

1. P_1 sends chq_i^1 to P_i for $i \in M$, and each P_i unmasks the string written in the auxiliary information field of the received cheque.
2. For each $i \in M$, P_i tells r'_i to P_1 by sending chq_1^i, and P_1 unmasks $r'_2 \oplus \cdots \oplus r'_{n-1} \oplus \mathsf{chq}_n^1$ using the received values.
3. P_1 sends chq_n^1 to P_n, and P_n unmasks the string written in the auxiliary information fields of the received cheque. (As a result, P_1 learns $\{\mathsf{chq}_i^n\}_{i\in[n-1]}$.)
4. P_n tells y to P_i by sending chq_i^n for each $i \in [n-1]$.

Output exchange phase (some parties behave maliciously):

- If corrupted P_1 aborts the protocol at step 3.
  - For each $i \in M$, P_i submits chq_1^i to $\mathcal{F}_{\mathrm{bank}}$.
- If corrupted P_1 submits $\{\mathsf{chq}_i^1\}_{i\in\hat{H}}$ to $\mathcal{F}_{\mathrm{bank}}$, where $\hat{H} \subseteq M$.
  - P_j submits chq_1^j to $\mathcal{F}_{\mathrm{bank}}$ for $j \in \hat{H}$. If P_1 further submits chq_n^1, P_n submits $\{\mathsf{chq}_i^n\}_{i\in[n-1]}$, and $\{P_i\}_{i\in M\setminus\hat{H}}$ submit $\{\mathsf{chq}_1^i\}_{i\in M\setminus\hat{H}}$ to the bank.
- If corrupted P_n aborts the protocol or submits $\{\mathsf{chq}_i^n\}_{i\in\hat{H}}$ to $\mathcal{F}_{\mathrm{bank}}$, $\hat{H} \subseteq [n-1]$ in step 4. (It includes the case where an adversary tells y to only some of honest parties.)
  - P_1 submits $\{\mathsf{chq}_j^1\}_{j\in M}$ to $\mathcal{F}_{\mathrm{bank}}$.
  - For each $j \in M$, P_j submits chq_1^j to $\mathcal{F}_{\mathrm{bank}}$.
- If corrupted $\{P_i\}_{i\in C_1}$ abort the protocol at step 2 and corrupted $\{P_i\}_{i\in C_2}$ submit $\{\mathsf{chq}_1^i\}_{i\in C_2}$ to $\mathcal{F}_{\mathrm{bank}}$, where $C_1, C_2 \subseteq M$. (To consider cases of malicious behaviors, we suppose that C_1 and C_2 are never empty sets at the same time.)
  - In the cases of $(C_1 = C_2) \vee (C_1 = \emptyset)$, P_1 submits $\{\mathsf{chq}_i^1\}_{i\in M}$ and chq_n^1. (Note that, in this case, P_1 can compute chq_n^1 using $\{r'_i\}_{i\in M}$.) Further, $\{P_i\}_{i\in M\setminus C_1\cup C_2}$ submit $\{\mathsf{chq}_1^i\}_{i\in M\setminus C_1\cup C_2}$ and P_n submits $\{\mathsf{chq}_i^n\}_{i\in[n-1]}$ to $\mathcal{F}_{\mathrm{bank}}$.
  - In the case of $(C_1 \neq C_2) \wedge (C_1 \neq \emptyset)$, P_1 submits $\{\mathsf{chq}_i^1\}_{i\in M}$, and $\{P_i\}_{i\in M\setminus C_1\cup C_2}$ submit $\{\mathsf{chq}_1^i\}_{i\in M\setminus C_1\cup C_2}$ to $\mathcal{F}_{\mathrm{bank}}$.

learning the output. Also, it does not require that each honest party receives the same compensation. The property of the non-equivalence of compensation is needed to prove the security of our protocols, and we leave an open problem to construct a protocol satisfying the equivalence of compensation as in the two-party case.

Functionality 4 is a formal description for secure *multi-party* computation with legally enforceable fairness. In the input phase, the functionality $\mathcal{F}_{n,f}^{\geq\alpha}$ receives inputs for f. Moreover, it allows the simulator $\mathcal{S}$ to specify a subset H' of honest parties. The subset captures compensated parties. The simulator can choose how to pay the penalties from corrupted parties' balances.

The output phase depends on $h' = |H'|$. If $h' = 0$, all parties can learn the output, and no party is penalized. The case of $0 < h' < h$ captures the cases where an adversary aborts the protocol without learning the output. Note that not all honest parties receive compensation in this case. In the case of $h' = h$, the functionality allows the simulator to learn the output first. Afterwards, the simulator can choose whether or not to abort the protocol without telling the output to honest parties by replying fair or unfair, like Functionality 3. This step allows the simulator to re-designate how to pay the penalties from corrupted parties' balances. We use H'' to capture the cases where an adversary tells only some of honest parties the output.

Definition 2 Let π be a protocol and let f be a multi-party functionality. We say that a protocol π securely computes f with α-legally enforceable fairness if π securely compute the functionality $\mathcal{F}_{n,f}^{\geq\alpha}$ according to Definition 1.

4.2 Proposed protocol I: $O(n)$ rounds and $O(n\alpha)$ fees

We first present our n-party protocol that achieves $O(n)$ rounds and $O(n\alpha)$ fees. Hereafter, we denote by chq_j^i a cheque for payment from P_j to P_i.

Before presenting formal description, we informally give the idea behind of the proposed protocol I: In the main computation phase, parties run secure multi-party computation, and only P_1 receives a cheque chq_2^1, as well as the two-party protocol. The cheque chq_2^1 has the recursive structure such as chq_2^1 holds chq_3^2 that holds chq_4^3, and so on. The deepest one holds the desired value $y = f(x_1, \ldots, x_n)$ and each chq_{i+1}^i is masked with a random value generated by P_i. Thus, to learn y, parties need to unmask the cheques sequentially in the output exchange phase. (We defer to discuss security in the case of occurring malicious behaviour later in this subsection.)

We present the formal description in Protocol 5. Since the ideal oblivious transfer is sufficient to realize the main computation phase with constant rounds and the output exchange phase requires n rounds, the protocol requires $O(n)$ rounds.

Also, P_n is the party that requires the largest balance at the beginning of the protocol. The balance is $\$(n-1)\alpha$, and thus the protocol requires $O(n\alpha)$ fees. To summarize the result, we can derive the following theorem.

Theorem 1 *For every n-party functionality f there exists a protocol that securely computes f with α-legally enforceable fairness in the $(\mathcal{F}_{\mathrm{OT}}, \mathcal{F}_{\mathrm{CA}}, \mathcal{F}_{\mathrm{bank}})$-hybrid model. The protocol requires $O(n)$ rounds and $O(n\alpha)$ fees.*

To present an intuitive understanding of the security property, we here describe each case of the output exchange phase when some parties behave maliciously. (Appendix A shows the formal proof.)

Security intuition: Let us consider the case where corrupted P_j $(2 \leq j \leq n)$ aborts, i.e., he/she does not send chq^j_{j+1} to P_{j+1}. This case corresponds to the first item of the output exchange phase (some parties behave maliciously). Since we want to focus on the case where fairness may be violated, we suppose that P_j colludes with $P_{j+1}, \ldots, P_n$. Note that otherwise, the adversary cannot learn the output. Then, every honest party submits his/her cheque to the bank. That is, honest P_i sends chq^i_{i+1} to $\mathcal{F}_{\mathrm{bank}}$. As a result, every honest party gets $\$\alpha$ as compensation, i.e., the protocol achieves legally enforceable fairness in this case. Note that if P_1 aborts, no one has published the random strings, the adversary cannot steal the output, and it is not a case of giving compensation.

Next, we describe the case where a malicious party submits his/her cheque to the bank, which is the second item. We discuss this case separately for the cases where P_j $(1 \leq j \leq n-1)$ submits the cheque or P_n submits the cheque. If corrupted P_j submits chq^j_{j+1} to the bank and takes $\$j\alpha$ from P_{j+1}. Since we want to focus on the case where fairness may be violated, we suppose that P_{j+1} is honest. $\mathcal{F}_{\mathrm{bank}}$ sends a copy of the cheque to the payer, and P_{j+1} learns R_j from the cheque and gets chq^{j+1}_{j+2}. Then, P_{j+1} submits the cheque to the bank and gets $\$(j+1)\alpha$ from P_{j+2}. Then, P_{j+2} can learn the cheque chq^{j+2}_{j+3} in the similar way. Parties repeat this procedure until P_n learns $\{\mathsf{chq}^n_i\}_{i\in[n-1]}$ and submits them to the bank. As a result, since all cheques are submitted to the bank, parties' balances return to the initial state and all parties learn the output y from the cheques of P_n. If a corrupted party in $P_{j+1} \ldots, P_n$ refuses to submit his/her cheque, the output is not revealed to honest parties. In this case, honest parties get $\$\alpha$ as compensation by submitting cheques. Thus, the protocol achieves legally enforceable fairness in this case. Suppose corrupted P_n submits $\{\mathsf{chq}^n_i\}_{i\in H}$ to the bank and takes $\$\alpha$ from each honest party. Note that all honest parties must already hold their cheques before P_n gets the submitted cheques. Thus, each honest party can get back $\$\alpha$ by submitting cheques to the bank. Since all honest parties learn the output from P_n's cheques, the protocol also achieves legally enforceable fairness in this case.

4.3 Proposed protocol II: $O(1)$ rounds and $O(n^2\alpha)$ fees

We next present our n-party protocol that achieves $O(1)$ rounds and $O(n^2\alpha)$ fees. Before presenting formal description, we informally give the idea to achieve constant rounds: In the main computation phase, parties run secure multi-party computation, and only P_1 receives cheques, which consist of unmasked and masked ones. We now focus on the masked cheque chq^1_n that holds $\{r_{n,i} \oplus \mathsf{chq}^n_i\}_{i\in[n-1]}$ in the auxiliary information field, where each chq^n_i holds the output $y = f(x_1, \ldots, x_n)$. The random strings $\{r_{n,i}\}_{i\in[n-1]}$ are generated by P_n, and chq^1_n is masked with $r'_2 \oplus \cdots \oplus r'_{n-2}$, where each r'_k is generated by P_k. Namely, chq^1_n has a two-tiered structure. That is, parties need to unmask this cheque two times to learn y, in the output exchange phase. On the first unmasking, $P_2, \ldots, P_n$ send their random strings to P_1, and P_1 unmasks the cheque using the $r'_2 \oplus \cdots \oplus r'_{n-2}$, and P_1 learns $\{r_{n,i} \oplus \mathsf{chq}^n_i\}_{i\in[n-1]}$. On the second unmasking, P_1 sends $\{r_{n,i} \oplus \mathsf{chq}^n_i\}_{i\in[n-1]})$ to P_n, and P_n unmasks these cheques using $\{r_{n,i}\}_{i\in[n-1]}$ and learns y.

See Protocol 6, which shows the formal description of our protocol. Since the ideal oblivious transfer is sufficient to achieve the main computation phase with constant rounds and the output exchange phase is realized with only four rounds, this protocol is performed with constant rounds. Also, P_1 is the party that requires the largest balance at the beginning of the protocol. The balance is $\$((n-1)(n-2)+1)\alpha$, and the protocol requires $O(n^2\alpha)$ fees. To summarize the result, we can derive the following theorem.

Theorem 2 *For every n-party functionality f there exists a protocol that securely computes f with α-legally enforceable fairness in the $(\mathcal{F}_{\mathrm{OT}}, \mathcal{F}_{\mathrm{CA}}, \mathcal{F}_{\mathrm{bank}})$-hybrid model. The protocol requires $O(1)$ rounds and $O(n^2\alpha)$ fees.*

Here, we give a security intuition of Theorem 2. (Appendix B shows the formal proof.) Below, as in Sect. 4.2, we describe each case of the output exchange phase when some parties behave maliciously. Hereafter, let $M := \{2, \ldots, n-1\}$.

Security intuition: First, let us consider the cases where P_1 behaves maliciously. If corrupted P_1 aborts the protocol in step 3, honest P_i submits chq^i_1 to $\mathcal{F}_{\mathrm{bank}}$ for each $i \in M$. Note that P_i has unmasked his/her cheque in step 2 for all $i \in M$. As a result, every honest party obtains $\$\alpha$ or more as compensation. Thus, the protocol achieves legally enforceable fairness in this case.

If corrupted P_1 submits $\{\mathsf{chq}^1_i\}_{i\in\hat{H}}$ to $\mathcal{F}_{\mathrm{bank}}$, where $\hat{H} \subseteq M$, then P_j submits chq^j_1 to $\mathcal{F}_{\mathrm{bank}}$ for $j \in \hat{H}$. As a result, every honest party obtains positive money. If P_1 further submits chq^1_n to the bank, P_n submits $\{\mathsf{chq}^n_i\}_{i\in[n-1]}$ and $\{P_i\}_{i\in M\setminus\hat{H}}$ to the bank. As a result, the balances of all honest parties become initial states since parties use all cheques.

Since we confirmed honest parties do not lose money and learn the output value y from cheque chq_i^n, the protocol achieves legally enforceable fairness in this case.

Next, let us discuss the cases where corrupted P_n behaves maliciously. If P_n aborts the protocol or submits $\{\mathsf{chq}_i^n\}_{i\in H'}$ to $\mathcal{F}_{\text{bank}}$, $H' \subseteq [n-1]$ in step 4, then P_1 submits $\{\mathsf{chq}_j^1\}_{j\in M}$ to $\mathcal{F}_{\text{bank}}$ and P_j submits chq_1^j to $\mathcal{F}_{\text{bank}}$ for each $j \in M$. The balances of parties who learn the output become initial states, and parties who do not learn the output receive compensation. Thus, the protocol achieves legally enforceable fairness in this case.

Finally, let us consider the case where some of $\{P_j\}_{j\in M}$ behave maliciously. We discuss this case separately for the cases where (i) $C_1 = C_2 \vee C_1 = \emptyset$ or (ii) $C_1 \neq C_2 \wedge C_1 \neq \emptyset$. In case (i), P_1 submits $\{\mathsf{chq}_j^1\}_{j\in M}$ and chq_n^1, and $\{P_j\}_{j\in M\setminus C_1}$ submit $\{\mathsf{chq}_j^1\}_{j\in M\setminus C_1}$ to $\mathcal{F}_{\text{bank}}$. Further, P_n obtains $\{\mathsf{chq}_i^n\}_{i\in[n-1]}$ from chq_n^1 and submits the cheques to the bank. As a result, the balances of all honest parties become initial states since parties use all cheques.

In case (ii), P_1 submits $\{\mathsf{chq}_j^1\}_{j\in M}$, and $\{P_i\}_{i\in M\setminus C_1\cup C_2}$ submit $\{\mathsf{chq}_1^i\}_{i\in M\setminus C_1\cup C_2}$ to $\mathcal{F}_{\text{bank}}$. We note that P_1 cannot use chq_n^1 in this case since there is a party in $\{P_j\}_{j\in M}$ who does not reveal the random value. Thus, we need to make sure that P_1 can get compensation even if he/she cannot use chq_n^1. Let $\hat{M} := M \setminus C_1$ be the set of parties who submit their cheques to the bank, i.e., cheques $\{\mathsf{chq}_i\}_{i\in\hat{M}}$ are submitted to the bank. P_1 obtains $\$(n-2)^2\alpha$ by using $\{\mathsf{chq}_j^1\}_{j\in M}$ and loses $\$(n-1)\hat{m}\alpha$, where $\hat{m} = |\hat{M}|$. Noting that the maximum value of $\hat{m}$ is $n-3$, it needs to satisfy that $(n-2)^2 > (n-1)(n-3)$ for that P_1 gets compensation without using chq_n^1. Since the inequality satisfies for arbitrary positive integer n, we confirmed that P_1 gets compensation for any $\hat{m} \in [n-3]$. Thus, the protocol achieves legally enforceable fairness in this case too.

Remark 2 The reason why the payment amounts of $\{\mathsf{chq}_j^1\}_{j\in M}$ and $\{\mathsf{chq}_1^j\}_{j\in M}$ are $\$(n-2)\alpha$ and $\$(n-1)\alpha$, respectively, comes from the last case (ii) in the security intuition discussion. We show the derivation process: Let q_1 and q_m be the payment amounts of $\{\mathsf{chq}_j^1\}_{j\in M}$ and $\{\mathsf{chq}_1^j\}_{j\in M}$, respectively. In order that P_1 does not lose money even if he/she cannot use chq_n^1, the total amount of money P_1 receives by using $\{\mathsf{chq}_j^1\}_{j\in M}$ needs to be larger than the total amount of money he/she loses by $n-3$ cheques in $\{\mathsf{chq}_1^j\}_{j\in M}$. (Note that P_1 can use chq_n^1 if all of $\{\mathsf{chq}_1^j\}_{j\in M}$ are submitted to the bank.) It means that $q_1(n-2) > q_m(n-3)$ needs to hold. Further, in order that each of $P_2, \ldots, P_{n-1}$ can receive compensation, $q_m > q_1$ must hold since the difference $q_m - q_1$ is his/her compensation. Since it is sufficient that $q_m = q_1 + 1$ holds, we can derive the payment amounts from the inequality $q_1(n-2) > (q_1+1)(n-3)$. The least solution of this inequality is $q_1 = n-2$.

Functionality 7 The trusted bank functionality $\mathcal{F}_{\text{bank}}^t$ with expiration time

Running with $\mathcal{F}_{\text{CA}}$ and parties $P_1, \ldots, P_n$. Let $\mathsf{bal}[u]$ ($u \in [n]$) be a variable denoting the current balance of P_u.

Setup phase: Initialize used and time as empty set and map, respectively. Send $(\mathsf{Retrieve}, u)$ to $\mathcal{F}_{\text{CA}}$ for every $u \in [n]$ and wait to the response. If any response has $\perp$, then terminates the functionality. Stores the keys $\{vk\}_{u\in[n]}$, where vk_u is the verification key retrieved for P_u.

Execution phase: Upon receiving a cheque $\mathsf{chq}(cid, i \to j, q, \tau, z)$ at a time ct, perform the following process.

1. Check if the cheque is valid and if $(cid, i, j) \notin \mathsf{used}$. If both checks are passed, continue the process. Otherwise ignore the cheque.
2. If $\tau < \mathsf{ct}$, continue to the next step. Otherwise, see the value of $\mathsf{time}(cid, \tau)$.
   - If $\mathsf{time}(cid, \tau) = \perp$ or $\mathsf{time}(cid, \tau) > \mathsf{ct}$, terminate the functionality.
   - Otherwise, i.e., $\mathsf{time}(cid, \tau) \leq \mathsf{ct}$, continue to the next step.
3. Set $\mathsf{bal}[i] = \mathsf{bal}[i] - q$ and $\mathsf{bal}[j] = \mathsf{bal}[j] + q$.
4. Set $\mathsf{used} \leftarrow \mathsf{used} \cup \{(cid, i, j)\}$ and $\mathsf{time}(cid, \tau) \leftarrow \mathsf{ct} + t$.
5. Send the cheque and $\mathsf{ct} + t$ to P_i.

5 Electronic cheque with expiration time

5.1 Modification of trusted bank functionality

Since the cheque discussed above does not have a expiration time, parties can use their cheques *indefinitely*. It implies that parties must hold the cheques forever to enjoy legally enforceable fairness. Since it is a heavy burden for parties, we here tackle this issue. Henceforth, we suppose that all parties (and the trusted bank) are synchronous, and they can watch the same wall clock.

To resolve the problem, we first modify the definition of the cheque so that it has a field to specify an expiration time. The modified format is $\mathsf{chq}(cid, i \to j, q, \tau, z)$, where τ refers to a positive integer representing the expiration time and the other parameters remain unchanged.

We further adjust the trusted bank functionality to reflect the above change. We note that it is not enough to reject expired cheques simply. This is because our protocols with such a trusted bank functionality does not satisfy the legally enforceable fairness. For instance, in Protocol 5, suppose that the expiration times of all cheques are τ_{ex}, which is a time long after the beginning of the protocol. Then, an adversary who corrupts P_1 can violate fairness as follows: After P_1 receives chq_2^1 as the main computation result, the adversary waits for time to pass without proceeding with the procedure.

Functionality 8 Secure multi-party computation with legally enforceable fairness using time limits $\mathcal{F}^{\geq\alpha}_{n,f,(\tau_{in},\tau_{out})}$

Running with parties $P_1, \ldots, P_n$ and a simulator $\mathcal{S}$ that corrupts parties $\{P_j\}_{j\in C}$. $\mathsf{bal}[i]$ denotes the current balance of P_i for $i \in [n]$.

Input phase: Wait to receive the following messages.

- (input, x_i) from P_i for $i \in H$
- $(\mathsf{input}, \{x_j\}_{j\in C}, H', \{\alpha_i\}_{i\in H'}, \{\beta_j\}_{j\in C})$ from $\mathcal{S}$, where $H' \subseteq H$, β_j is a non-negative integer, and $\sum_{i\in H'} \alpha_i = \sum_{j\in C} \beta_j$. (Let ct_{in} be the time when the functionality received the inputs from $\mathcal{S}$.)

If some input is invalid, some α_i is less than α, or $\mathsf{ct}_{in} > \tau_{in}$, then send $\perp$ to all parties and terminate the functionality.

Output phase: Compute $y \leftarrow f(x_1, \ldots, x_n)$.

1. Perform the following process depending on h', where $h' := |H'|$.
   - If $h' = 0$, then send y to P_i for $i \in [n]$, and terminate.
   - If $0 < h' < h$, then set $\mathsf{bal}[i] = \mathsf{bal}[i] + \alpha_i$ for all $i \in H'$ and $\mathsf{bal}[j] = \mathsf{bal}[j] - \beta_j$ for $j \in C$, and terminate.
   - If $h' = h$, then send message y to $\mathcal{S}$.
2. In the case of $h' = h$, wait to $\mathcal{S}$'s response.
   - If $\mathcal{S}$ replies fair before τ_{out}, send y to P_i for $i \in H$.
   - If $\mathcal{S}$ replies $(\mathsf{unfair}, H'', \{\alpha'_i\}_{i\in H}, \{\beta'_j\}_{j\in C})$ before τ_{out}, perform the following process, where $H'' \subsetneq H$, β'_j is a non-negative integer, $\alpha'_i \geq \alpha$ for $i \in H \setminus H''$, and $\sum_{i\in H} \alpha'_i = \sum_{j\in C} \beta'_j$.
     - Send y to P_i for $i \in H''$.
     - Set $\mathsf{bal}[i] = \mathsf{bal}[i] + \alpha'_i$ for $i \in H$ and $\mathsf{bal}[j] = \mathsf{bal}[j] - \beta'_j$ for $j \in C$.
   - If $\mathcal{S}$ does not reply a valid response before τ_{out}, set $\mathsf{bal}[i] = \mathsf{bal}[i] + \alpha_i$ for $i \in H$ and $\mathsf{bal}[j] = \mathsf{bal}[j] - \beta_j$ for $j \in C$.

Just before τ_{ex}, the adversary submits chq^1_2 to the bank and gets money. Although the other parties can get back money by using their cheques, it may be impossible due to the lack of time remaining. As a result, the adversary obtains money and violates fairness.

To circumvent the above attack, we allow the bank to extend expiration times. Precisely, when a party uses a cheque, the bank extends the expiration times of corresponding other cheques. The extension of time is determined with a parameter t. We present the formal description in Functionality 7. See steps 2 and 4 in the execution phase. When a party submits a cheque before the expiration time τ, the bank extends the expiration time of other cheques to $\max(\tau, \mathsf{ct}+t)$, where ct is the current time. If we set t as a sufficient time to submit a cheque, the protocol circumvents the above attack.

5.2 Our protocols based on the modified functionality

We can apply the modified bank functionality to both of our proposed protocols, i.e., Protocols 5 and 6, with only a few changes. Before presenting the changes, we set the expiration time τ of the cheques and the extension time parameter t: The expiration time of all cheques is set to be a sufficiently large integer, denoted by τ_{ex}, so that the protocol can complete before the time in the case where all parties are honest. The parameter t is also set to be sufficiently large to use a cheque. We suppose that all parties agree on these values before beginning of the protocol.

The only changes to the protocol procedures are to set appropriate time limits for some steps. We first set the time limit for the main computation phase. Let τ_{main} be a sufficiently large integer time such that the main computation phase finishes before the time. Parties conclude that the protocol is aborted by an adversary if the main computation phase does not finish before τ_{main}. Next, we set the time limits for the output exchange phase. Let ρ denote the round number of the output exchange phase in the case where all parties behave honestly, i.e., $\rho = n$ and $\rho = 4$ for Protocols 5 and 6, respectively. We set the time limits $(\tau_1, \ldots, \tau_\rho)$ to the output exchange phase, where τ_i is the time limit for round i and $\tau_\rho = \tau_{ex}$. The interval between τ_i and τ_{i+1} must be sufficiently large to complete the procedure for that round. If a party does not complete his/her procedure before the time limit, the other parties conclude that the party has aborted and use their cheques to restore fairness as in the protocols, described in Sect. 4.

Remark 3 Our formalization allows the bank to *re-extend* expiration times: If the expiration time τ is extended to $\mathsf{ct}_1 + t$ and then a party submits a cheque with expiration time τ at ct_2 $(< \mathsf{ct}_1 + t)$, the bank again extends the period to $\mathsf{ct}_2 + t$. This fact allows an adversary to extend a expiration time maliciously. More specifically, it can repeatedly extend periods by making cheques whose payer is a corrupted party and continuing to submit them to the bank. However, note that parties cannot use multiple cheques with the same (cid, i, j) due to the list used in Functionality 7. Thus, an adversary can extend the time at most $(n-1)^2 t$,[1] and thus the protocol terminates at most before the time $\tau_{ex} + (n-1)^2 t$.

To accurately capture the functionality of the proposed protocols based on the modified cheque, we describe a new functionality, Functionality 8. It adds two parameters (τ_{in}, τ_{out}) to Functionality 4, where τ_{in} and τ_{out} are the time limits for simulator's input and response, respectively. For our protocols, it is sufficient to set the parameters as $\tau_{in} = \tau_{main}$ and $\tau_{out} = \tau_{ex} + (n-1)^2 t$. Below, we introduce modified legally enforceable fairness.

Definition 3 Let π be a protocol and let f be a multi-party functionality. We say that a protocol π securely computes f

[1] This value is calculated from the fact that the number of corrupted parties is at most $n-1$. That is, there are $n-1$ possible payers of a valid cheque that the adversary can make. Since there are $n-1$ possible payees for each payer, the adversary can make $(n-1)^2$ valid cheques.

with $(\alpha, \tau_{in}, \tau_{out})$-legally enforceable fairness if π securely compute the functionality $\mathcal{F}^{\geq\alpha}_{n,f,(\tau_{in},\tau_{out})}$ according to Definition 1.

To summarise, we can derive the following theorem.

Theorem 3 *Suppose that all parties can watch the same clock. For every n-party functionality f there exists a protocol that securely computes f with $(\alpha, \tau_{in}, \tau_{out})$-legally enforceable fairness in the $(\mathcal{F}_{\mathrm{OT}}, \mathcal{F}_{\mathrm{CA}}, \mathcal{F}^{t}_{\mathrm{bank}})$-hybrid model.*

We omit the proof since it is almost the same as the proofs of Theorems 1 and 2. Note that the simulator can watch the clock and knows the time limits set in the protocol. Hence, the simulator can determine whether the adversary has aborted or not according to the response times of the adversary.

6 Conclusion

This paper focused on secure computation with legally enforceable fairness that achieves fairness by imposing a monetary penalty on an adversary. Lindell [23] introduced the trusted bank functionality and formalized secure computation with legally enforceable fairness based on the functionality. Further, he showed a general protocol with legally enforceable fairness for any functionality. However, his formalization and protocol are applicable only to the two-party setting.

We formalized the legally enforceable fairness applicable to an arbitrary number of parties based on the trusted bank functionality as well as [23]. Furthermore, we proposed two protocols achieving secure multi-party computation with legally enforceable fairness. The first protocol achieves $O(n)$ rounds and $O(n\alpha)$ fees, where n is the number of parties, and α is a parameter for the penalty amount. The second one achieves $O(1)$ rounds and $O(n^2\alpha)$ fees. Moreover, we introduced a cheque with validity period and new bank functionality that can deal with such cheques. We showed how to apply the modified cheque and bank to our proposed protocols.

As mentioned in Sect. 1.2, the cryptocurrency-based solution is the mainstream in achieving fairness with monetary penalties. Such a line of works proposed more advanced applications: covert security with monetary penalties [12, 31] and secure cash distribution [5, 8, 11, 21]. The bank-based solution may also reach such advanced applications, and we hope that this work leads to them.

Appendix A Security Proof for Proposed Protocol I

This section presents a proof of Theorem 1. Hereafter, for a finite set X of positive integers, $\max(X)$ and $\min(X)$ denote the maximum and minimum element of X, respectively. Let $\mathcal{A}$ be a (real-world) adversary corrupting $\{P_i\}_{i\in C}$. We partition the set of corrupted parties C as $C = C_1 \sqcup \cdots \sqcup C_\mu$ such that each C_i consists of consecutive elements $C_i = \{\min(C_i), \min(C_i)+1, \ldots, \min(C_i)+|C_i|\}$ for $1 \leq i \leq \mu$ and $\max(C_i) < \min(C_{i+1})$ for $1 \leq i \leq \mu - 1$. For example, when $C = \{1, 2, 5, 7, 8, 9, 10\}$, we partition the set into $C_1 = \{1, 2\}$, $C_2 = \{5\}$, $C_3 = \{7, 8, 9, 10\}$.

Algorithm 9 Cheque simulation for C_*, where $1 \in C_*$

Input: A set of identifiers C_*, a unique identifier cid, signing keys, and random strings $\{r_i\}_{i\in C_*\setminus\{1\}}$.

Output: $\mathsf{chq}^1_2 = \mathsf{chq}(cid, 2 \to 1, \alpha, R_2)$.

- If $|C_*| = 1$, R_2 is a random string with the appropriate length.
- If $|C_*| > 1$, for $i \in \{2, \ldots, |C_*|\}$, $R_i = r_i \oplus \mathsf{chq}^i_{i+1}$, where $\mathsf{chq}^i_{i+1} = \mathsf{chq}(cid, i+1 \to i, i\alpha, R_{i+1})$ and $R_{|C_*|+1}$ is a random string with the appropriate length.

Algorithm 10 Cheque simulation for C_*, where $1, n \notin C_*$

Let $c_{\min} := \min(C_*)$ and $c_{\max} := \max(C_*)$.

Input: A set of identifiers C_*, a unique identifier cid, signing keys, and random strings $\{r_i\}_{i\in C_*}$.

Output: $\mathsf{chq}^{c_{\min}-1}_{c_{\min}}$, where $\mathsf{chq}^{c_{\min}-1}_{c_{\min}} = \mathsf{chq}(cid, c_{\min} \to c_{\min} - 1, (c_{\min}-1)\alpha, R_{c_{\min}})$.

- If $|C_*| = 1$, $R_{c_{\min}}$ is a random string with the appropriate length.
- If $|C_*| > 1$, for $i \in \{c_{\min}, \ldots, c_{\max}\}$, $R_i = r_i \oplus \mathsf{chq}^i_{i+1}$, $\mathsf{chq}^i_{i+1} = \mathsf{chq}(cid, i+1 \to i, i\alpha, R_{i+1})$, and $R_{c_{\max}+1}$ is a random string with the appropriate length.

Algorithm 11 Cheque simulation for C_*, where $n \in C_*$

Let $c_{\min} := \min(C_*)$.

Input: A set of identifiers C_*, a unique identifier cid, signing keys, random strings $\{r_{n,i}\}_{i\in[n-1]}$, and the output y. If $|C_*| > 2$, input $\{r_i\}_{i\in C_*\setminus\{n\}}$ further.

Output: $\mathsf{chq}^{c_{\min}-1}_{c_{\min}}$, where $\mathsf{chq}^{c_{\min}-1}_{c_{\min}} = \mathsf{chq}(cid, c_{\min} \to c_{\min} - 1, (c_{\min}-1)\alpha, R_{c_{\min}})$.

- If $|C_*| = 1$, $R_{c_{\min}} = \{r_{n,i} \oplus \mathsf{chq}^n_i\}_{i\in[n-1]}$, where $\mathsf{chq}^n_i := \mathsf{chq}(cid, i \to n, \alpha, y)$.
- If $|C_*| > 1$, $R_{c_{\min}} = r_{c_{\min}} \oplus \mathsf{chq}^{c_{\min}}_{c_{\min}+1}$. For $i \in \{c_{\min}, \ldots, n-1\}$, $\mathsf{chq}^i_{i+1} := \mathsf{chq}(cid, i+1 \to i, i\alpha, R_{i+1})$ and $R_n := \{r_{n,i} \oplus \mathsf{chq}^n_i\}_{i\in[n-1]}$

Formally, the main computation phase realizes the following functionality.

Input: For each $j \in [n]$, P_j inputs $((vk_j, sk_j), \{vk_i\}_{i\in[n]\setminus\{j\}}, (x_j, \mathsf{rand}_j), \alpha, \lambda, cid_j)$, where $\mathsf{rand}_1 = \bot$, $\mathsf{rand}_i = r_i$ for $i \in \{2, \ldots, n-1\}$, and $\mathsf{rand}_n = \{r_{n,i}\}_{i\in[n-1]}$.
Output: P_1 receives chq_1^2 and the other parties receive nothing. The property of chq_1^2 is as in the protocol.

We suppose that this functionality is achieved according to Definition 1 under the $\mathcal{F}_{\mathrm{OT}}$-hybrid model.

We construct a simulator $\mathcal{S}$ as follows.

1. $\mathcal{S}$ invokes $\mathcal{A}$ with its inputs $\{x_i\}_{i\in C}$, a security parameter λ, and a penalty amount parameter α.
2. $\mathcal{S}$ generates a key-pair $(vk'_i, sk'_i) \leftarrow \mathsf{Gen}(1^\lambda)$ for $i \in H$, records the key-pairs, and reply to $\mathcal{A}$ whenever $\mathcal{A}$ sends a query intended for $\mathcal{F}_{\mathrm{CA}}$ as follows:
   - If $\mathcal{A}$ sends $(\mathsf{Register}, P_j, vk'_j)$ intended for $\mathcal{F}_{\mathrm{CA}}$, $\mathcal{S}$ records vk'_j if $j \in C$.
   - If $\mathcal{A}$ sends $(\mathsf{Retrieve}, P_i)$ intended for $\mathcal{F}_{\mathrm{CA}}$, $\mathcal{S}$ replies $(\mathsf{Retrieve}, P_i, vk'_i)$.
3. $\mathcal{S}$ gets $\mathcal{A}$'s inputs $\{((vk_j, sk_j), \{vk_i\}_{i\in[n]\setminus\{j\}}, (x_j, \mathsf{rand}_j), \alpha, \lambda, cid_j)\}_{j\in C}$ sent for the functionality of the main computation phase, where $\mathsf{rand}_1 = \bot$, $\mathsf{rand}_i = r_i$ for $i \in \{2, \ldots, n-1\}$, and $\mathsf{rand}_n = \{r_{n,i}\}_{i\in[n-1]}$. If some key differs from the key chosen in the previous step, $\mathcal{S}$ sends an invalid input to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.
4. $\mathcal{S}$ generates $cid_i \in_{\mathrm{R}} \{0,1\}^\lambda$ for $i \in H$ and sets $cid = cid_1 \| \cdots \| cid_n$.
5. If $1 \in C$, $\mathcal{S}$ runs Algorithm 9 for C_1 and sends $\mathcal{A}$ the output. Then, it waits the message from the adversary.
   - If $\mathcal{S}$ receives $\mathsf{chq}_{\max(C_1)+1}^{\max(C_1)}$, $\mathcal{S}$ runs Algorithm 10 for C_2 and sends $\mathcal{A}$ the output. (If the received cheque is invalid, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ sends $\mathsf{chq}_{\max(C_1)+1}^{\max(C_1)}$ intended for the bank (in order to steal money from honest $P_{\max(C_1)+1}$), $\mathcal{S}$ runs Algorithm 10 for C_2 and sends $\mathcal{A}$ the output. (If the cheque $\mathcal{A}$ sent is invalid, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ aborts, $\mathcal{S}$ sends an invalid input to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.
6. If $1 \notin C$, $\mathcal{S}$ runs Algorithm 10 for C_1, and sends $\mathcal{A}$ the output.
7. For $i = 2, \ldots, \mu-1$, $\mathcal{S}$ works depending on $\mathcal{A}$'s response as follows:
   - If $\mathcal{S}$ receives $\mathsf{chq}_{\max(C_i)+1}^{\max(C_i)}$, $\mathcal{S}$ runs Algorithm 10 for C_{i+1} and sends $\mathcal{A}$ the output. (If the received cheque is invalid, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ sends $\mathsf{chq}_{\max(C_i)+1}^{\max(C_i)}$ intended for the bank (in order to steal money from honest $P_{\max(C_i)+1}$), $\mathcal{S}$ runs Algorithm 10 for C_{i+1} and sends $\mathcal{A}$ the output. (If the cheque $\mathcal{A}$ sent is invalid, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ aborts, $\mathcal{S}$ sends $(\mathsf{input}, \{x_i\}_{i\in C}, H', \{\alpha_i\}_{i\in H'}, \{\beta_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, where H' is the set of honest parties whose identifiers are less than $\min(C_i)$ and α_i and β_j are set as in the protocol. Afterward, it terminates the simulation.

   Below, consider the case where the simulator receives valid $\mathsf{chq}_{\max(C_{\mu-1})+1}^{\max(C_{\mu-1})}$ from the adversary.
8. If $n \notin C$, $\mathcal{S}$ runs Algorithm 10 for C_μ and sends $\mathcal{A}$ the output. It waits for $\mathcal{A}$'s response.
   - If $\mathcal{S}$ receives $\mathsf{chq}_{\max(C_\mu)+1}^{\max(C_\mu)}$, $\mathcal{S}$ sends $(\mathsf{input}, \{x_i\}_{i\in C}, \emptyset, \bot, \bot)$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, and learns y by the response. (If the received cheque is invalid, $\mathcal{S}$ ignores it.) It further creates $\{\mathsf{chq}_i^n\}_{i\in C}$ as the protocol and sends the cheques to $\mathcal{A}$.
   - If $\mathcal{A}$ sends $\mathsf{chq}_{\max(C_\mu)+1}^{\max(C_\mu)}$ intended for the bank (in order to steal money from honest $P_{\max(C_\mu)+1}$), $\mathcal{S}$ creates $\{\mathsf{chq}_i^n\}_{i\in C}$ as the protocol and sends $\{\mathsf{chq}_i^n\}_{i\in C}$ and $\{\mathsf{chq}_{\min(C_i)}^{\min(C_i)-1}\}_{i\in\{2,\ldots,\mu\}}$ to $\mathcal{A}$. (If the cheque $\mathcal{A}$ sent is invalid, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ aborts, $\mathcal{S}$ sends $(\mathsf{input}, \{x_i\}_{i\in C}, H', \{\alpha_i\}_{i\in H'}, \{\beta_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, where H' is the set of honest parties whose identifiers are less than $\min(C_i)$ and α_i and β_j are set as in the protocol. Afterward, it terminates the simulation.
9. If $n \in C$, $\mathcal{S}$ sends $(\mathsf{input}, \{x_i\}_{i\in C}, H, \{\alpha_i\}_{i\in H}, \{\beta_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, where α_i and β_j are set as in the protocol, and learns y by the response. Afterward, $\mathcal{S}$ runs Algorithm 11 for C_μ and sends $\mathcal{A}$ the output. It waits for $\mathcal{A}$'s response.
   - If $\mathcal{A}$ responds chq_i^n for all $i \in H$, $\mathcal{S}$ sends fair to $\mathcal{F}_{n,f}^{\geq\alpha}$.
   - If $\mathcal{A}$ sends $\{\mathsf{chq}_i^n\}_{i\in H}$ intended for the bank, $\mathcal{S}$ sends fair to $\mathcal{F}_{n,f}^{\geq\alpha}$. Further, it sends $\{\mathsf{chq}_{\min(C_j)}^{\min(C_j)-1}\}_{j\in\{2,\ldots,\mu\}}$ to $\mathcal{A}$.
   - If $\mathcal{A}$ sends $\{\mathsf{chq}_i^n\}_{i\in H''}$ intended for the bank, where $H'' \subsetneq H$, $\mathcal{S}$ sends $(\mathsf{unfair}, H'', \{\alpha'_i\}_{i\in H}, \{\beta'_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$. $\{\alpha'_i\}_{i\in H}$ and $\{\beta'_j\}_{j\in C}$ consist of the same values to the protocol. (Note that $\alpha'_i = 0$ for $i \in H''$ and $\alpha'_j \geq \alpha$ for $j \in H \setminus H''$.) Furthermore, it sends $\{\mathsf{chq}_{\min(C_j)}^{\min(C_j)-1}\}_{j\in\{2,\ldots,\mu\}}$ to $\mathcal{A}$.
   - If $\mathcal{A}$ responds $\{\mathsf{chq}_i^n\}_{i\in H''}$, where $H'' \subsetneq H$, i.e., the adversary sends the cheques to only some of honest parties, then $\mathcal{S}$ sends $(\mathsf{unfair}, H'', \{\alpha'_i\}_{i\in H}, \{\beta'_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, where $\{\alpha'_i\}_{i\in H}$ and $\{\beta'_j\}_{j\in C}$ consist of the same values to the protocol. (Note that $\alpha'_i \geq \alpha$ for all

$i \in H$.) Furthermore, $\mathcal{S}$ sends $\{\mathsf{chq}_{\min(C_i)}^{\min(C_i)-1}\}_{i\in\{2,\ldots,\mu\}}$ to $\mathcal{A}$.
- If $\mathcal{A}$ aborts, $\mathcal{S}$ sends $(\mathsf{unfair}, \emptyset, \{\alpha'_i\}_{i\in H}, \{\beta'_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, where $\{\alpha'_i\}_{i\in H}$ and $\{\beta'_j\}_{j\in C}$ consist of the same values to the protocol. Furthermore, $\mathcal{S}$ creates $\{\mathsf{chq}_i^{i-1}\}_{i\in C}$ and sends $\mathcal{A}$ the cheques.

When $\mathcal{S}$ terminates the simulation, it outputs whatever $\mathcal{A}$ outputs. We complete making up the simulation. $\mathcal{A}$'s view in the simulation is identical to the one's view in the hybrid execution of Protocol 5. □

Appendix B Security Proof for Proposed Protocol II

This section presents a proof of Theorem 2. We note that it generally follows the proof shown in Appendix A.

Formally, the main computation phase realizes the following functionality.

Input: For each $j \in [n]$, P_j inputs $((vk_j, sk_j), \{vk_i\}_{i\in[n]\setminus\{j\}}, (x_j, \mathsf{rand}_j), \alpha, \lambda, cid_j)$, where $\mathsf{rand}_1 = \perp$, $\mathsf{rand}_i = (r_i, r'_i)$ for $i \in \{2, \ldots, n-1\}$, and $\mathsf{rand}_n = \{r_{n,i}\}_{i\in[n-1]}$.

Algorithm 12 Cheque simulation for C, where $1 \in C$

Input: A set of identifiers C, a unique identifier cid, signing keys, and random strings $\{(r_i, r'_i)\}_{M\cap C}$.

Output: $(\{\mathsf{chq}_i^1\}_{i\in M}, R_{\mathsf{chq}})$, where $\mathsf{chq}_i^1 = \mathsf{chq}(cid, i \to 1, (n-2)\alpha, \ell_i)$.

- If $M \subset C$, then the random string $R_{\mathsf{chq}} = r'_2 \oplus \cdots \oplus r'_{n-1} \oplus \mathsf{chq}_n^1$, where $\mathsf{chq}_n^1 = \mathsf{chq}(cid, n \to 1, (n-1)\alpha, \{R_j\}_{j\in[n-1]})$ and each R_j is a random string with the appropriate length. Otherwise, R_{chq} is a random string with the appropriate length.
- If $i \in C$, then $\ell_i = r_i \oplus \mathsf{chq}_1^i$, where $\mathsf{chq}_1^i = \mathsf{chq}(cid, i \to 1, (n-1)\alpha, r'_i)$. Otherwise, ℓ_i is a random string with the appropriate length.

Algorithm 13 Cheque simulation for C, where $1 \in C$ and $M \not\subset C$

Input: A set of identifiers C, a unique identifier cid, signing keys, and random strings R_{chq} and $\{(r_i, r'_i)\}_{M\cap C}$. If $n \in C$, input $\{r_{n,j}\}_{j\in[n-1]}$ and y further.

Output: $\{\mathsf{chq}_1^i\}_{i\in M\setminus C}$, where $\mathsf{chq}_1^i = \mathsf{chq}(cid, 1 \to i, (n-1)\alpha, \ell_i)$.

- The random strings $\{\ell_i\}_{i\in M\setminus C}$ fulfill $R_{\mathsf{chq}} \oplus (\bigoplus_{i\in M\setminus C} \ell_i) \oplus (\bigoplus_{i\in M\cap C} r'_i) = \mathsf{chq}_n^1$ and $\mathsf{chq}_n^1 = \mathsf{chq}(cid, n \to 1, (n-1)\alpha, \{R_j\}_{j\in[n-1]})$.
- If $n \notin C$, each R_j is a random string with the appropriate length. Otherwise, $R_j = \mathsf{chq}_j^n \oplus r_{n,j}$, where chq_j^n is set as in the protocol.

Output: P_1 receives $\{\mathsf{chq}_i^1\}_{i\in M}$ and $r'_2 \oplus \cdots \oplus r'_{n-1} \oplus \mathsf{chq}_n^1$. (The properties of the cheques are as in the protocol.) The other parties receive nothing.

We suppose that this functionality is achieved according to Definition 1 under the $\mathcal{F}_{\mathrm{OT}}$-hybrid model.

We construct a simulator $\mathcal{S}$ as follows.

1. $\mathcal{S}$ invokes $\mathcal{A}$ with its inputs $\{x_i\}_{i\in C}$, a security parameter λ, and a penalty amount parameter α.
2. $\mathcal{S}$ generates a key-pair $(vk'_i, sk'_i) \leftarrow \mathsf{Gen}(1^\lambda)$ for $i \in H$, records the key-pairs, and reply to $\mathcal{A}$ whenever $\mathcal{A}$ sends a query intended for $\mathcal{F}_{\mathrm{CA}}$ as follows:
   - If $\mathcal{A}$ sends $(\mathsf{Register}, P_j, vk'_j)$ intended for $\mathcal{F}_{\mathrm{CA}}$, $\mathcal{S}$ checks if $j \in C$ and records vk'_j.
   - If $\mathcal{A}$ sends $(\mathsf{Retrieve}, P_i)$ intended for $\mathcal{F}_{\mathrm{CA}}$, $\mathcal{S}$ replies $(\mathsf{Retrieve}, P_i, vk'_i)$.
3. $\mathcal{S}$ gets $\mathcal{A}$'s inputs $\{((vk_j, sk_j), \{vk_i\}_{i\in[n]\setminus\{j\}}, (x_j, \mathsf{rand}_j), \alpha, \lambda, cid_j)\}_{j\in C}$ sent for the functionality of the main computation phase, where $\mathsf{rand}_1 = \perp$, $\mathsf{rand}_i = (r_i, r'_i)$ for $i \in \{2, \ldots, n-1\}$, and $\mathsf{rand}_n = \{r_{n,i}\}_{i\in[n-1]}$. If some key differs from the key chosen in the previous step, $\mathcal{S}$ sends an invalid input to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.
4. S generates $cid_i \in_{\mathrm{R}} \{0,1\}^\lambda$ for $i \in H$ and sets $cid = cid_1 \| \cdots \| cid_n$.

From step 5 and onward, we describe the proof in four separate cases: (I) $1 \in C \wedge n \in C$, (II) $1 \in C \wedge n \notin C$, (III) $1 \notin C \wedge n \in C$, (IV) $1 \notin C \wedge n \notin C$.

Case I: $1 \in C \wedge n \in C$

5. $\mathcal{S}$ runs Algorithm 12 and sends $\mathcal{A}$ the output $(\{\mathsf{chq}_i^1\}_{i\in M}, R_{\mathsf{chq}})$.[2] $\mathcal{S}$ waits the response from the adversary.
   - If $\mathcal{S}$ receives $\{\mathsf{chq}_i^1\}_{i\in M\cap H}$, $\mathcal{S}$ sends $(\mathsf{input}, \{x_i\}_{i\in C}, H, \{\alpha_i\}_{i\in H}, \{\beta_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, where α_i and β_j are set as in the protocol, and learns y by the response. (If there is an invalid cheque in the received cheques, $\mathcal{S}$ ignores the message.) Afterwards, it runs Algorithm 13 and sends $\mathcal{A}$ the output. Proceed to the next step.
   - If $\mathcal{A}$ sends $\{\mathsf{chq}_i^1\}_{i\in M\cap H}$ intended for the bank, $\mathcal{S}$ creates $\{\mathsf{chq}_1^i\}_{i\in M\cap H}$ and sends $\mathcal{A}$ them. Then, it terminates the simulation. (If there is an invalid cheque in the cheques $\mathcal{A}$ sent, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ aborts, $\mathcal{S}$ sends an invalid input to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.

[2] In case I, $M \not\subset C$ holds since at least one party is honest. Hence, R_{chq} is generated as a random string.

6. $\mathcal{S}$ waits to the response.[3]
   - If $\mathcal{A}$ responds chq_i^n for all $i \in H$, $\mathcal{S}$ sends fair to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.
   - If the adversary aborts, $\mathcal{S}$ sends (unfair, $\emptyset$, $\{\alpha_i'\}_{i\in H}$, $\{\beta_j'\}_{j\in C}$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, where $\{\alpha_i'\}_{i\in H}$ and $\{\beta_j'\}_{j\in C}$ as in the protocol. Afterwards, it creates $\{\mathsf{chq}_i^1\}_{i\in M\cap C}$ and sends $\mathcal{A}$ them. Then, $\mathcal{S}$ terminates the simulation.

Case II: $1 \in C \wedge n \notin C$

5. $\mathcal{S}$ runs Algorithm 12 and sends $\mathcal{A}$ the output ($\{\mathsf{chq}_i^1\}_{i\in M}$, R_{chq}).
6. Case of $M \subset C$, i.e., only P_n is honest: $\mathcal{S}$ waits the response from the adversary.
   - If $\mathcal{S}$ receives chq_n^1, it checks the validity. If it is invalid, $\mathcal{S}$ ignores the message. Otherwise, $\mathcal{S}$ sends (input, $\{x_i\}_{i\in C}$, $\emptyset$, $\perp$, $\perp$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, and learns y by the response. It further creates $\{\mathsf{chq}_i^n\}_{i\in C}$ as the protocol and sends the cheques to $\mathcal{A}$. Then, it terminates the simulation.
   - If $\mathcal{A}$ sends chq_n^1 intended for the bank, $\mathcal{S}$ creates $\{\mathsf{chq}_i^n\}_{i\in C}$ and sends $\mathcal{A}$ them. Then, it terminates the simulation. (If the cheque $\mathcal{A}$ sent is invalid, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ aborts, $\mathcal{S}$ sends an invalid input to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.
7. Case of $M \not\subset C$: $\mathcal{S}$ waits the response from the adversary.
   - If $\mathcal{S}$ receives $\{\mathsf{chq}_i^1\}_{i\in M\cap H}$, $\mathcal{S}$ runs Algorithm 13 and sends $\mathcal{A}$ the output. (If there is an invalid cheque in the received cheques, $\mathcal{S}$ ignores the message.) Proceed to the next step.
   - If $\mathcal{A}$ sends $\{\mathsf{chq}_i^1\}_{i\in M\cap H}$ intended for the bank, $\mathcal{S}$ creates $\{\mathsf{chq}_1^i\}_{i\in M\cap H}$ and sends $\mathcal{A}$ them. Then, it terminates the simulation. (If there is an invalid cheque in the cheques $\mathcal{A}$ sent, $\mathcal{S}$ ignores it.)
   - If $\mathcal{A}$ aborts, $\mathcal{S}$ sends an invalid input to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.
8. $\mathcal{S}$ waits to the response.
   - If $\mathcal{A}$ responds chq_n^1, $\mathcal{S}$ sends (input, $\{x_i\}_{i\in C}$, $\emptyset$, $\perp$, $\perp$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, and learns y by the response. It further creates $\{\mathsf{chq}_i^n\}_{i\in C}$ as the protocol and sends the cheques to $\mathcal{A}$. Then, it terminates the simulation.
   - If the adversary aborts, $\mathcal{S}$ sends (input, $\{x_i\}_{i\in C}$, H', $\{\alpha_i\}_{i\in H'}$, $\{\beta_j\}_{j\in C}$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, where H' is the set of honest parties whose identifiers are less than n, and α_i and β_j are set as in the protocol. $\mathcal{S}$ creates $\{\mathsf{chq}_1^i\}_{i\in M\cap H}$ and sends $\mathcal{A}$ them. Then, it terminates the simulation.

Case III: $1 \notin C \wedge n \in C$

5. For $j \in M \cap C$, $\mathcal{S}$ makes $\mathsf{chq}_j^1 = \mathsf{chq}(cid, j \to 1, (n-2)\alpha, r_j \oplus \mathsf{chq}_1^j)$, where $\mathsf{chq}_1^j = \mathsf{chq}(cid, 1 \to j, (n-1)\alpha, r_j')$. It sends the cheques to $\mathcal{A}$, and waits the response. (In this step, we suppose $M \cap C \neq \emptyset$ for brevity.)
   - If $\mathcal{S}$ receives $\{\mathsf{chq}_1^i\}_{i\in M\cap C}$, $\mathcal{S}$ sends (input, $\{x_i\}_{i\in C}$, H, $\{\alpha_i\}_{i\in H}$, $\{\beta_j\}_{j\in C}$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, where α_i and β_j are set as in the protocol, and learns y by the response. (If there is an invalid cheque in the received cheques, $\mathcal{S}$ ignores the message.) Afterwards, $\mathcal{S}$ creates $\mathsf{chq}_n^1 = \mathsf{chq}(cid, n \to 1, (n-1)\alpha, \{r_{n,i} \oplus \mathsf{chq}_i^n\}_{i\in[n-1]})$ and sends $\mathcal{A}$ it. (Proceed to the next step.)
   - If $\mathcal{A}$ sends $\{\mathsf{chq}_1^i\}_{i\in M\cap C}$ intended for the bank, $\mathcal{S}$ sends (input, $\{x_i\}_{i\in C}$, H, $\{\alpha_i\}_{i\in H}$, $\{\beta_j\}_{j\in C}$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, where α_i and β_j are set as in the protocol, and learns y by the response. (If there is an invalid cheque in the cheques $\mathcal{A}$ sent, $\mathcal{S}$ ignores it.) Afterwards, $\mathcal{S}$ creates $\{\mathsf{chq}_i^1\}_{i\in M\cap C}$ and chq_n^1, and sends $\mathcal{A}$ them. (Proceed to the next step.)
   - If $\mathcal{A}$ aborts, $\mathcal{S}$ sends (input, $\{x_i\}_{i\in C}$, H', $\{\alpha_i\}_{i\in H'}$, $\{\beta_j\}_{j\in C}$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, where H' is the set of honest parties whose identifiers are less than n and α_i and β_j are set as in the protocol. It creates $\{\mathsf{chq}_i^1\}_{i\in M\cap C}$ and sends $\mathcal{A}$ them. Then, $\mathcal{S}$ terminates the simulation.
6. $\mathcal{S}$ waits to the response.
   - If $\mathcal{A}$ responds chq_i^n for all $i \in H$, $\mathcal{S}$ sends fair to $\mathcal{F}_{n,f}^{\geq\alpha}$ and terminates the simulation.
   - If the adversary aborts, $\mathcal{S}$ sends (unfair, $\emptyset$, $\{\alpha_i'\}_{i\in H}$, $\{\beta_j'\}_{j\in C}$) to $\mathcal{F}_{n,f}^{\geq\alpha}$, where $\{\alpha_i'\}_{i\in H}$ and $\{\beta_j'\}_{j\in C}$ as in the protocol. It creates chq_1^n and $\{\mathsf{chq}_i^1\}_{i\in M\cap C}$, and sends $\mathcal{A}$ them. Then, $\mathcal{S}$ terminates the simulation.

Case IV: $1 \notin C \wedge n \notin C$

5. For $j \in M \cap C$, $\mathcal{S}$ makes $\mathsf{chq}_j^1 = \mathsf{chq}(cid, j \to 1, (n-2)\alpha, r_j \oplus \mathsf{chq}_1^j)$, where $\mathsf{chq}_1^j = \mathsf{chq}(cid, 1 \to j, (n-1)\alpha, r_j')$. It sends the cheques to $\mathcal{A}$, and waits the response. (In this step, we suppose $M \cap C \neq \emptyset$ for simplicity.)
   - If $\mathcal{S}$ receives $\{\mathsf{chq}_1^i\}_{i\in M\cap C}$, $\mathcal{S}$ sends (input, $\{x_i\}_{i\in C}$, $\emptyset$, $\perp$, $\perp$) to $\mathcal{F}_{n,f}^{\geq\alpha}$ and learns the output y. (If there is an invalid cheque in the received cheques, $\mathcal{S}$ ignores the message.) Afterwards, $\mathcal{S}$ creates $\{\mathsf{chq}_i^n\}_{i\in C}$ as in

[3] Note that in this step, we omit the cases where the adversary submits its cheques to the bank or tells the output only some of honest parties for the sake of simplicity. These cases also can be simulated as in the proof of Theorem 1. This is also true for case III.

the protocol and sends them, and terminate the simulation.

– If $\mathcal{A}$ aborts, $\mathcal{S}$ sends $(\mathsf{input}, \{x_i\}_{i\in C}, H', \{\alpha_i\}_{i\in H'}, \{\beta_j\}_{j\in C})$ to $\mathcal{F}_{n,f}^{\geq\alpha}$, where H' is the set of honest parties whose identifiers are less than n and α_i and β_j are set as in the protocol. It creates $\{\mathsf{chq}_i^1\}_{i\in M\cap C}$ and sends $\mathcal{A}$ them. Then, $\mathcal{S}$ terminates the simulation.

For all cases, when $\mathcal{S}$ terminates the simulation, it outputs whatever $\mathcal{A}$ outputs. We complete making up the simulation. $\mathcal{A}$'s view in the simulation is identical to the one's view in the hybrid execution of Protocol 6. □

Acknowledgements This work was supported by JSPS KAKENHI Grant Numbers JP23K16880 and JP21K17702, and JST CREST Grant Number JPMJCR22M1.

Declarations

Compliance with Ethical Standards The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Andrychowicz, M., Dziembowski, S., Malinowski, D., Mazurek, Ł.: Fair two-party computations via bitcoin deposits. In: Financial Cryptography and Data Security. 105–121. Springer, Berlin Heidelberg (2014)
2. Asharov, G., Beimel, A., Makriyannis, N., Omri, E.: Complete characterization of fairness in secure two-party computation of boolean functions. In: Theory of Cryptography. 199–228. Springer, Berlin Heidelberg (2015)
3. Asokan, N., Schunter, M., Waidner M.: Optimistic protocols for fair exchange. In Proceedings of the 4th ACM Conference on Computer and Communications Security, CCS '97, 7-17. Association for Computing Machinery, (1997)
4. Bartoletti, M., Zunino, R.: Constant-deposit multiparty lotteries on bitcoin. In Financial Cryptography and Data Security, 231–247. Springer International Publishing, (2017)
5. Baum, C., David, B., Dowsley, R.: Insured mpc: Efficient secure computation with financial penalties. In Financial Cryptography and Data Security: 24th International Conference, FC 2020, 404-420. Springer-Verlag, (2020)
6. Beaver, D., Goldwasser, S.: Multiparty computation with faulty majority. In 30th Annual Symposium on Foundations of Computer Science, 468–473, (1989)
7. Bentov, I., Kumaresan, R.: How to use bitcoin to design fair protocols. In Advances in Cryptology – CRYPTO 2014, 421–439. Springer Berlin Heidelberg, (2014)
8. Bentov, I., Kumaresan, R., Miller, A.: Instantaneous decentralized poker. In Advances in Cryptology – ASIACRYPT 2017, 410–440. Springer International Publishing, (2017)
9. Canetti, R.: Universally composable signature, certification, and authentication. In Proceedings. 17th IEEE Computer Security Foundations Workshop, 2004., 219–233 (2004)
10. Cleve, R.: Limits on the security of coin flips when half the processors are faulty. In Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing, STOC '86, 364-369. Association for Computing Machinery, (1986)
11. David, B., Dowsley, R., Larangeira, M.: Kaleidoscope: An efficient poker protocol with payment distribution and penalty enforcement. In Financial Cryptography and Data Security, 500–519. Springer Berlin Heidelberg, (2018)
12. Faust, S., Hazay, C., Kretzler, D., Schlosser, B.: Financially backed covert security. In Public-Key Cryptography - PKC 2022 - 25th IACR International Conference on Practice and Theory of Public-Key Cryptography, Virtual Event, Proceedings, Part II, of Lecture Notes in Computer Science, **13178** 99–129. Springer, (2022)
13. Friolo, D., Massacci, F., Ngo, C.N., Venturi, D.: Cryptographic and financial fairness. IEEE Trans. Inform. Foren. Secur. **17**, 3391–3406 (2022)
14. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game. In Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing, STOC '87, 218-229. Association for Computing Machinery, (1987)
15. Goldwasser, S., Levin, L.: Fair computation of general functions in presence of immoral majority. In Advances in Cryptology-CRYPTO' 90, 77–93. Springer Berlin Heidelberg, (1991)
16. Gordon, S.D., Hazay, C., Katz, J., Lindell, Y.: Complete fairness in secure two-party computation. J. ACM **58**(6), 1–37 (2011)
17. Ishai, Y., Prabhakaran, M., Sahai, A.: Founding cryptography on oblivious transfer – efficiently. In Advances in Cryptology – CRYPTO 2008, 572–591. Springer Berlin Heidelberg, (2008)
18. Kiayias, A., Zhou, H.-S., Zikas,V.: Fair and robust multi-party computation using a global transaction ledger. In Proceedings, Part II, of the 35th Annual International Conference on Advances in Cryptology — EUROCRYPT 2016 - **9666**, 705-734. Springer-Verlag, (2016)
19. Kilian, J.: Founding crytpography on oblivious transfer. In Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing, STOC '88, 20-31. Association for Computing Machinery, (1988)
20. Kumaresan, R., Bentov, I.: Amortizing secure computation with penalties. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16, 418-429. Association for Computing Machinery, (2016)
21. Kumaresan, R. Moran, T., Bentov, I.: How to use bitcoin to play decentralized poker. In Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, CCS '15, 195-206. Association for Computing Machinery, (2015)
22. Kumaresan, R., Vaikuntanathan, V., Vasudevan, P. N.: Improvements to secure computation with penalties. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16, 406-417. Association for Computing Machinery, (2016)
23. Lindell, A. Y.: Legally-enforceable fairness in secure two-party computation. In Topics in Cryptology – CT-RSA 2008, 121–137. Springer Berlin Heidelberg, (2008)

24. Micali, S.: Secure protocols with invisible trusted parties. Weizmann Inst. of Science, In Workshop for Multi-Party Secure Protocols (1998)
25. Nakai, T., Shinagawa, K.: Secure Computation with Non-Equivalent Penalties in Constant Rounds. In 3rd International Conference on Blockchain Economics, Security and Protocols (Tokenomics 2021), volume 97 of Open Access Series in Informatics (OASIcs), 5:1–5:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, (2022)
26. Nakai, T., Shinagawa, K.: Constant-round linear-broadcast secure computation with penalties. Theor. Comput. Sci. **959**, 113874 (2023)
27. Nakai, T., Shinagawa, K.: Secure multi-party computation with legally-enforceable fairness. In Information and Communications Security, 161–178. Springer Nature Singapore, (2023)
28. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. Cryptography Mailing list at https://metzdowd.com, 03 (2009)
29. Uchizono, S., Nakai, T., Watanabe, Y., Iwamoto, M.: Constant-deposit multiparty lotteries on bitcoin for arbitrary number of players and winners. In Information Security and Cryptology – ICISC 2023, 133–156. Singapore, Springer Nature Singapore (2024)
30. Yao, A. C.: Protocols for secure computations. In Proceedings of the 23rd Annual Symposium on Foundations of Computer Science, 160–164, (1982)
31. Zhu, R. Ding,C., Huang, Y.: Efficient publicly verifiable 2pc over a blockchain with applications to financially-secure computations. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19, 633-650. Association for Computing Machinery, (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.